

LLM : Architectural Deep Dive

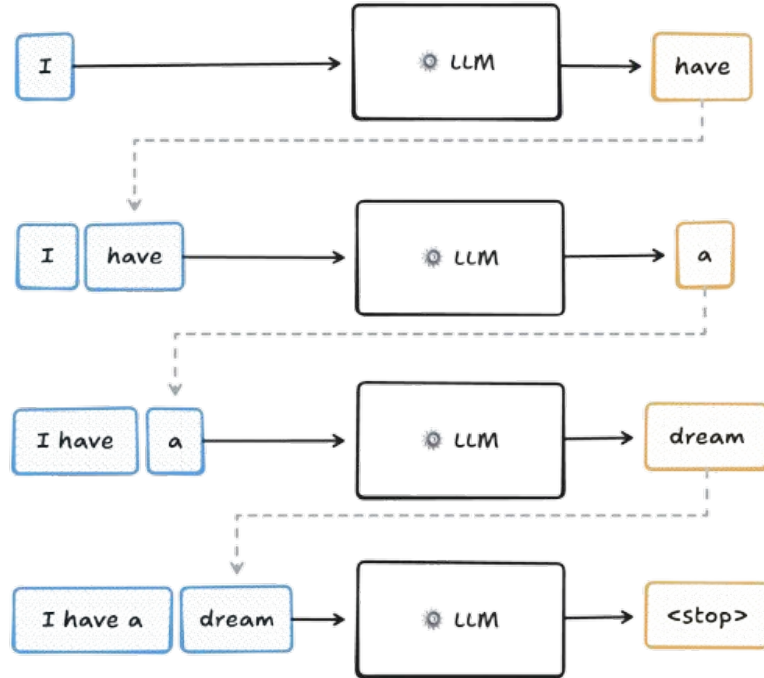
Understanding a High Throughput LLM Inference System

By: **Ayush Satyam**
Software Engineer at Red Hat
GitHub: **@ayushsatyam146**

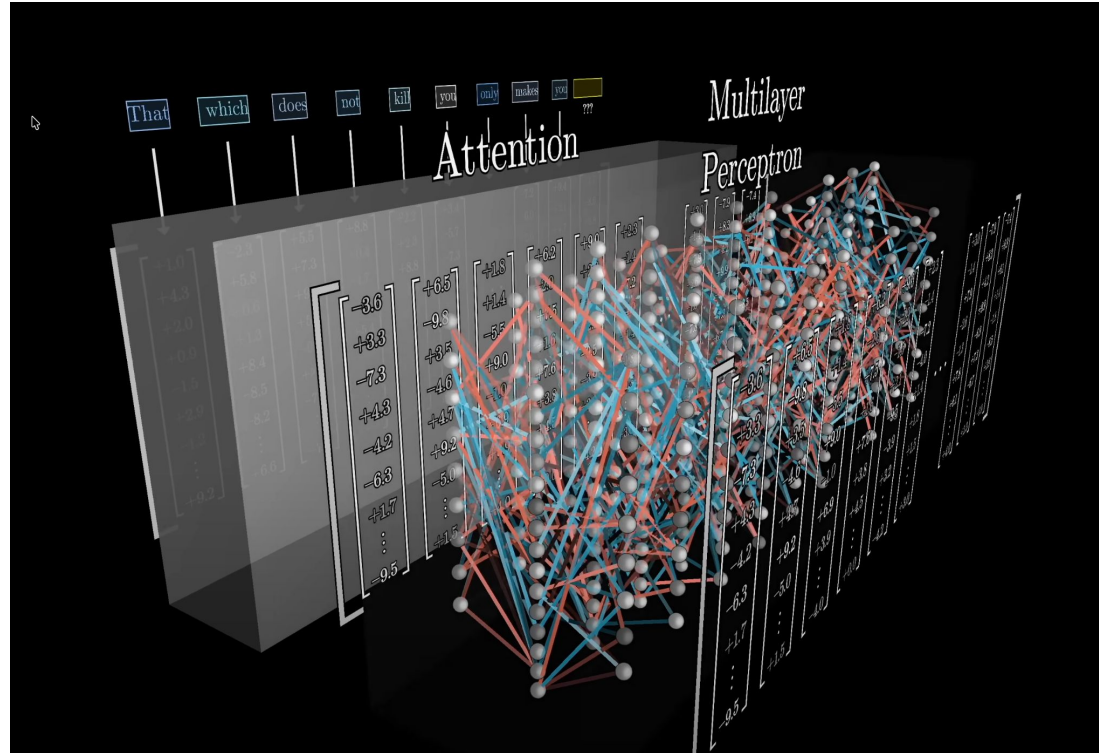
Serving LLMs is (surprisingly) expensive

- A ton of GPUs are needed to serve production scale LLM services
- Each GPU can serve only handful of requests
- GPU vRAM quickly becomes a painful bottleneck

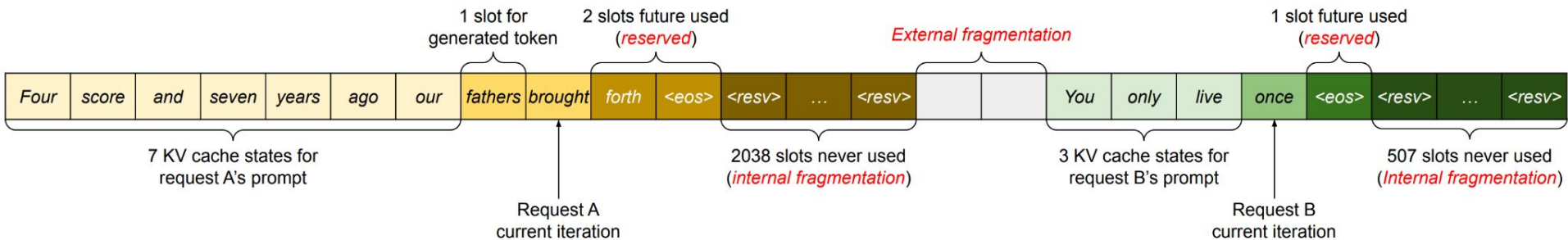
How LLM token generation works



Basic transformer architecture



Memory waste in KV cache



- **Internal fragmentation:** due to unknown output length
- **Reservation:** not used currently but will be used in future
- **External fragmentation:** due to different sequence lengths

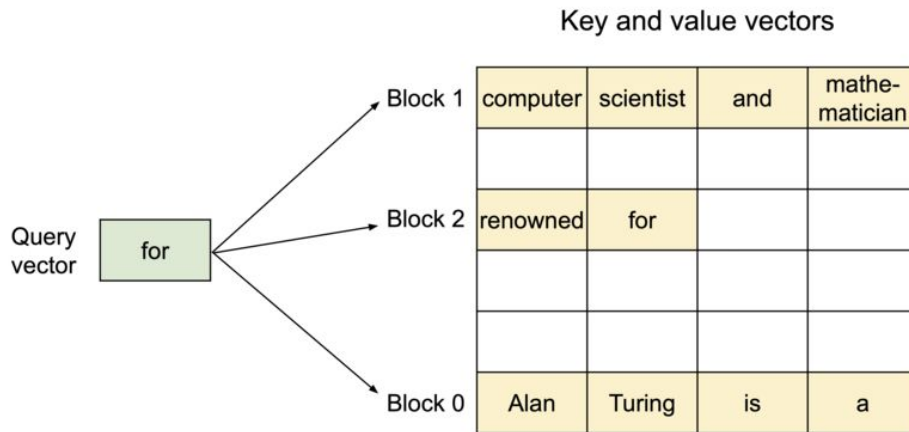
Only 20-40% memory utilization

Why vLLM is awesome!!

- Paged Attention
- Optimized K-V Caching
- Optimized CUDA kernels
- Speculative Decoding
- Chunked Prefill and many more

Paged Attention

An attention algorithm that allows for storing continuous keys and values in non-contiguous memory space.



Managing KV cache: Lessons from OS

Request
A

Alan	Turing	is	a
computer	scientist	and	mathema tician
renowned			

Logical KV blocks

Artificial	Intelli- gence	is	the
future	of	tech- nology	

Request
B

Block Table

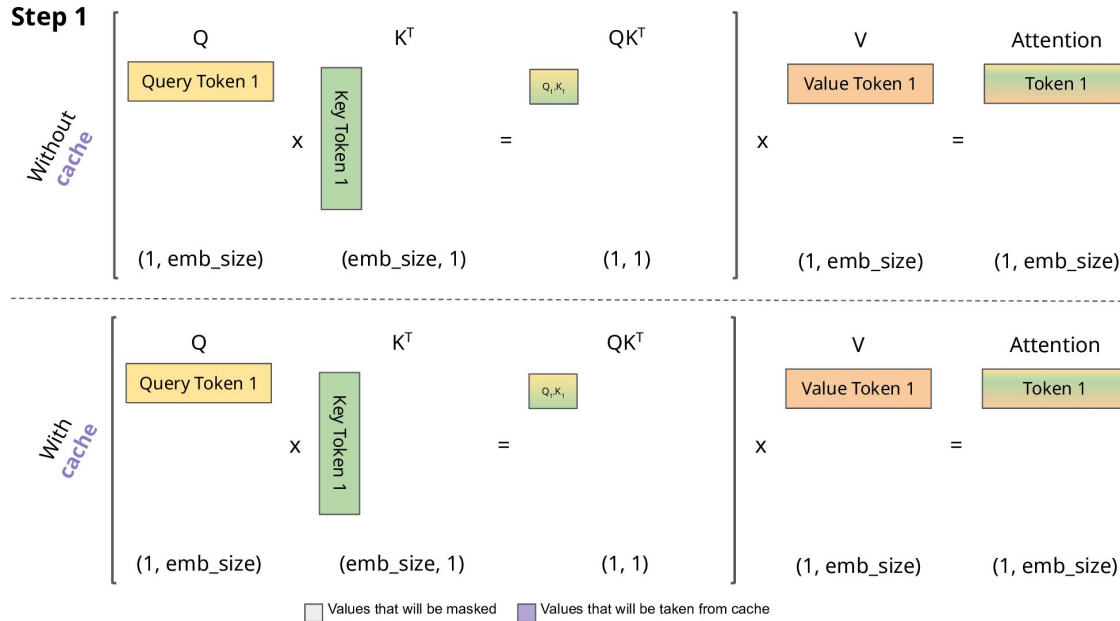
Block Table

Physical KV blocks

computer	scientist	and	mathe- matician
Artificial	Intelli- gence	is	the
renowned			
future	of	tech- nology	
Alan	Turing	is	a

KV Cache

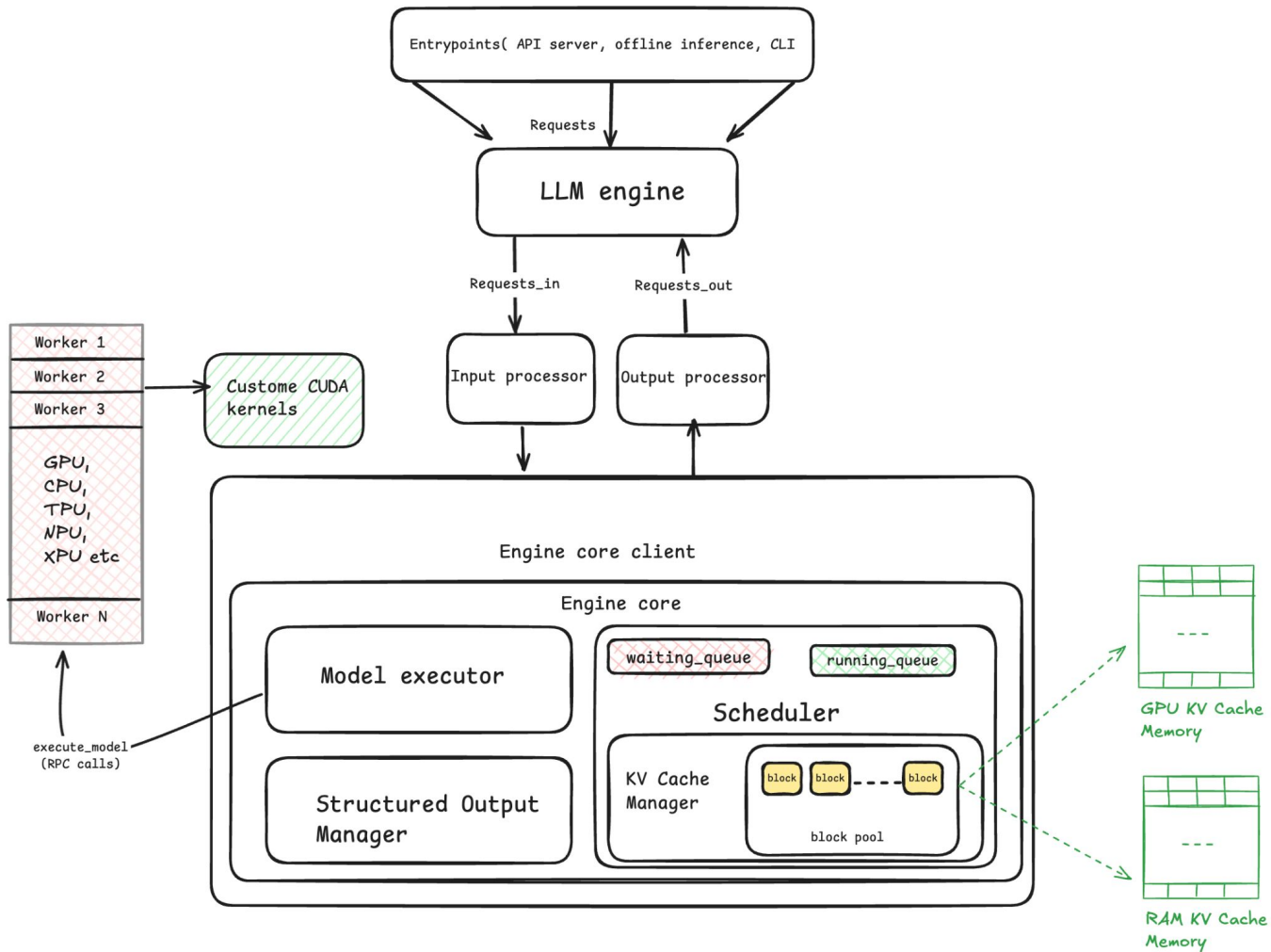
Caching Key and Value vectors in self-attention saves redundant computation and accelerates decoding

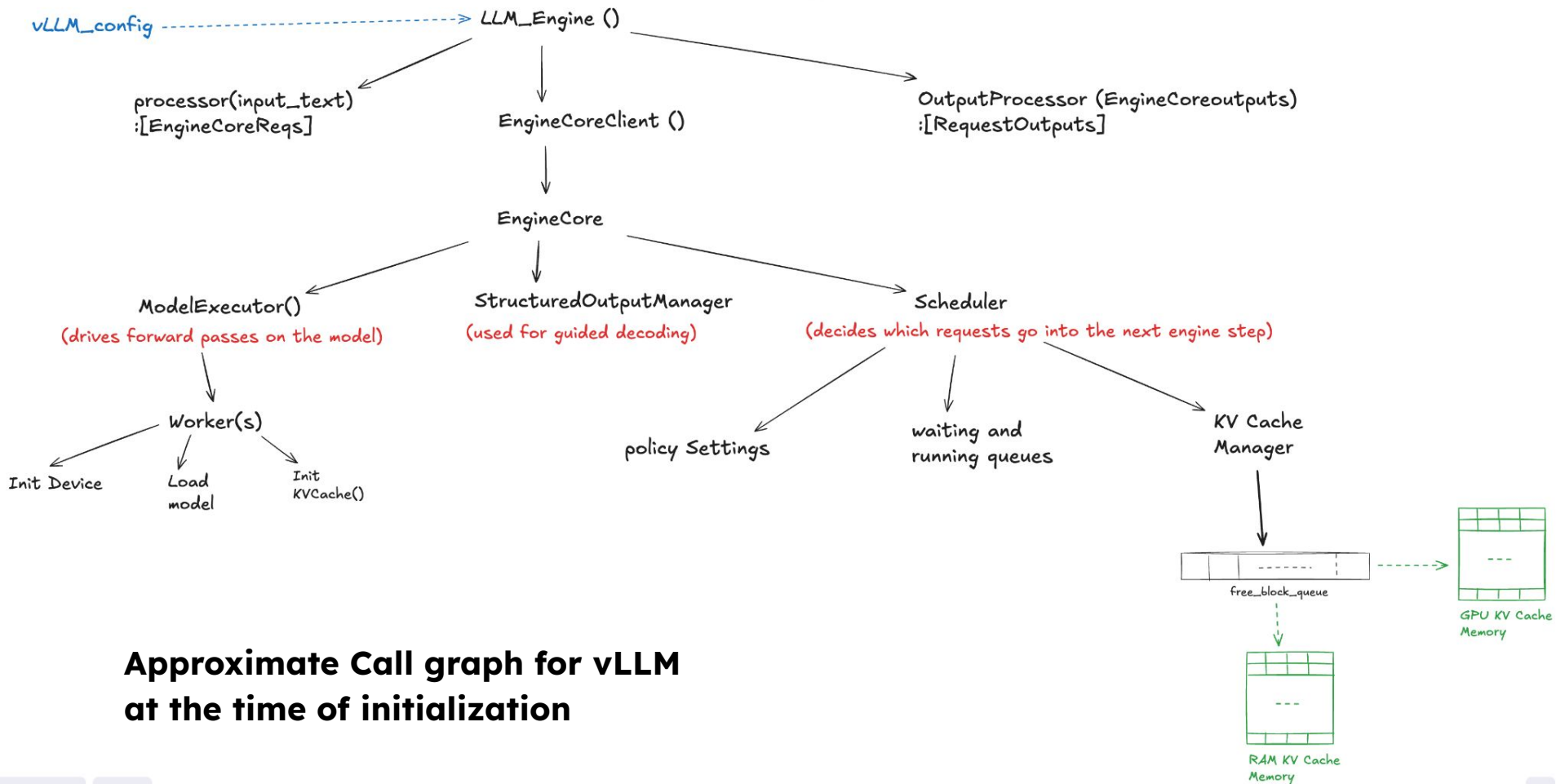


10k feet view of the system

What are the core components of the system?

- Tokenizer
- LLM Engine
- KV cache manager
- Worker
- Scheduler
- Model executor....
- ... Let's **SEE** in detail





Let's see some Advanced features

- Chunked Prefill
- Prefix Caching
- Guided decoding (FSM)
- Disaggregated PD
- and more...

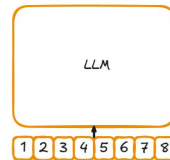
Chunked Prefill

Chunked Prefill is a technique for handling long prompts by splitting their prefill step into smaller chunks

Example:

```
long_prefill_token_threshold = 8 toks  
block_size = 4 toks  
prompt_token_ids = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18]
```

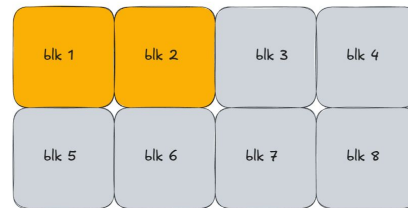
1st fwd pass



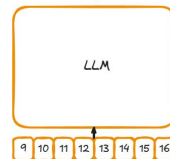
GPU

KV cache paged memory

after 1st fwd pass 2 blocks contain KV's (8 tokens)

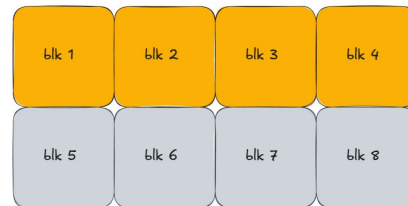


2nd fwd pass



GPU

after 2nd fwd pass 4 blocks contain KV's (16 tokens)

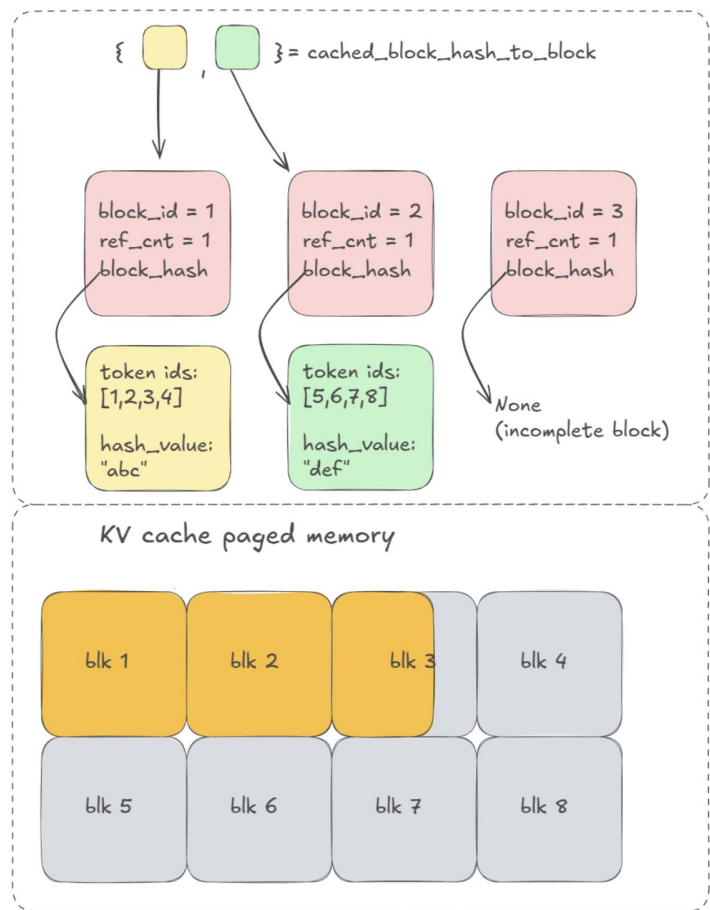


Prefix Caching

Prefix Caching avoids recomputing tokens that multiple prompts share at the beginning - hence prefix.

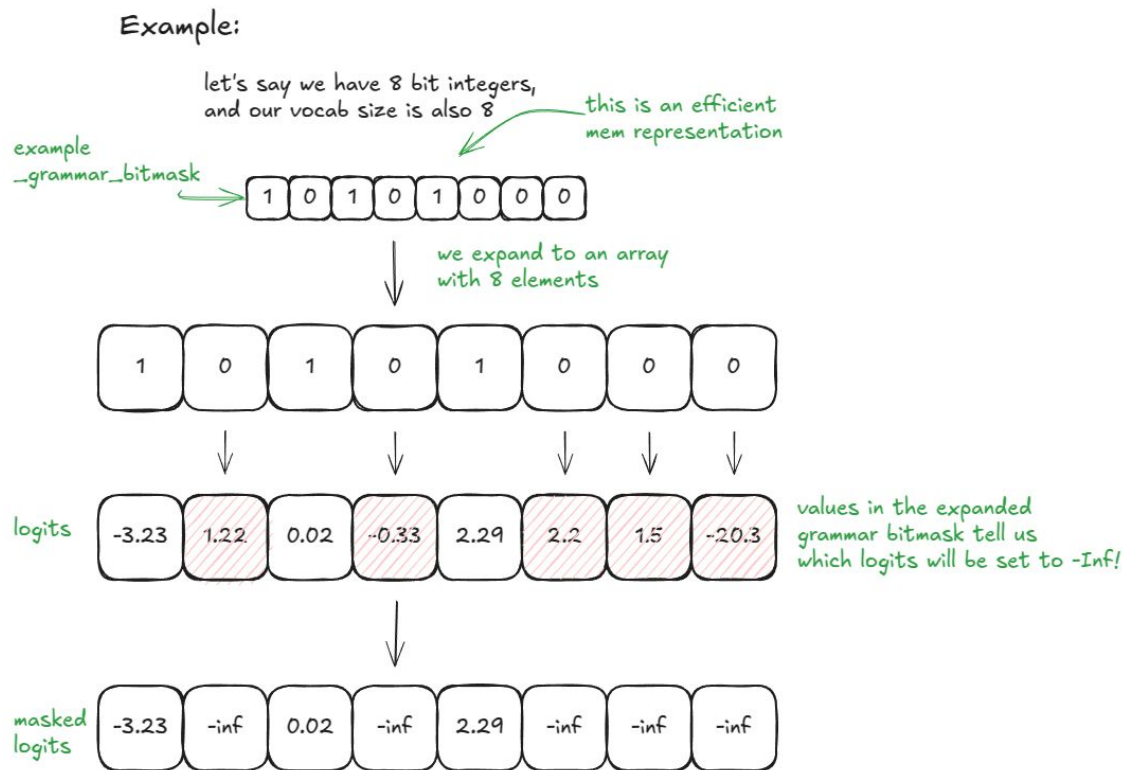


blocks saved by hashes for future requests



Guided Decoding

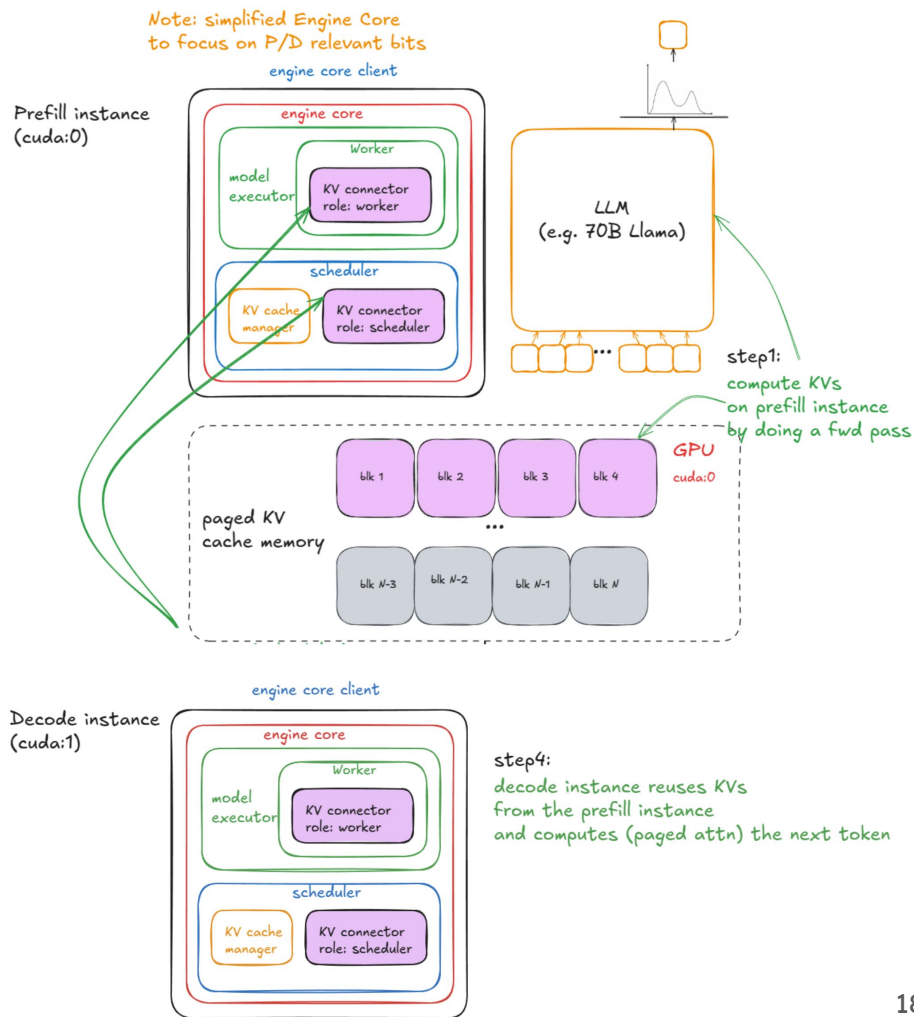
Guided Decoding is a technique where, at each decoding step, the logits are constrained by a grammar-based finite state machine.



Disaggregated PD

Prefill and decode have very different performance profiles (compute-bound vs. memory-bandwidth-bound), so separating their execution is a sensible design. It gives tighter control over latency — both TFTT

(time-to-first-token) and ITL (inter-token latency)



References:

- Anatomy of vLLM [Blog](#) (Must read!)
- vLLM [codebase](#) (Must read!)
- [Talk](#) by WooSuk Kwon & Zhuohan Li
- Modal notebook [here](#)
- vLLM paper [here](#)
- vLLM [meetups](#)
- 3 blue 1 brown **Neural networks** [playlist](#)

THANK YOU

Reach out to me on:



@ayushsatyam146



@ayushsatyam146

